Academic Integrity Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Market Basket Analysis: High-Performance Itemset Mining via a Distributed PySpark SON Pipeline

Algorithms for Massive Dataset

June 27, 2026

Abstract

This report outlines the design, implementation, and empirical evaluation of a distributed frequent itemset mining framework leveraging the Savasere, Omiecinski, and Navathe (SON) algorithm. Built on top of Apache Spark's low-level Resilient Distributed Dataset (RDD) API, the system bypasses relational high-level optimization layers to analyze casting correlations across two distinct data footprints. Benchmarks demonstrate that partitioning workloads minimizes network shuffle traffic during localized frequency generation passes, maintaining sub-linear scaling curves over a real-world transaction space of 98,835 organic rows.

1 Methodological Framework

Frequent Itemset Mining (FIM) at scale suffers from severe computational and memory overhead due to exponential candidate space explosion. While traditional Apriori models scale poorly because they require iterative global disk scans, the SON Algorithm implements an elegant, mathematically sound partitioning pipeline to enforce a strict two-pass MapReduce processing bound:

1. **Local Candidate Generation (Pass 1):** The global transactional matrix is segmented into k independent, non-overlapping RDD partitions. A localized Apriori function isolates and maps each chunk independently. To ensure no valid itemsets are dropped, the global support threshold s_{global} drops proportionally inside each worker node based on partition weight:
$$s_{local} = s_{global} \times \frac{|P_i|}{|D_{global}|} \tag{1}$$
An itemset that fails to satisfy s_{local} within its independent horizontal partition cannot mathematically achieve global support. This property prunes structural permutations at the localized worker level.

2. **Global Validation & Filtering (Pass 2):** The driver unifies the local worker outputs into a single distinct candidate matrix. To prevent heavy data-frame shuffling across the cluster network during global counting, this candidate dictionary is packed into a read-only Spark broadcast variable. Nodes evaluate true global item support in a single parallel reduction step.

2 Data Pipeline & Structural Organization

The processing framework evaluates two operational datasets to contrast development logic against macro scaling behaviors.

2.1 Development Sandbox (Dataset 1)

A synthetic baseline table modeling uniform Hollywood cast networks was assembled to verify programmatic flow correctness. By testing within a deterministic dataset, it was possible to manually trace exact support limits, checking that structural functions generated valid pairs without encountering edge-case drops or faulty combinations.

2.2 Kaggle Global Footprint (Dataset 2)

Production-scale evaluation utilized an organic IMDB compilation from Kaggle tracking 98,835 entries.

- **Target Transformations:** The analysis uses the `title` field as the unique transactional transaction basket identifier, while the comma-delimited `cast` record acts as the underlying item array.
- **Ingestion Strategy:** Source CSV data frames are read from a distributed file workspace (`big_imdb_data/*.csv`) and instantly stripped into underlying RDD arrays to distribute parallel chunk parsing across the cluster topology.

3 Applied Data Cleansing Mechanisms

Raw production metadata is inherently noisy and contains parser artifacts that degrade standard tokenization. The pipeline cleans strings directly inside the map stage using three primary operations:

1. **Padding Elimination:** Leading and trailing spacing variables are removed using Spark's `F.trim()` function. Null inputs, blank arrays, and text anomalies like literal `"None"` markers are dropped.
2. **Parser Cleanse Mapping:** JSON bracket wrappers (`[`, `]`) and literal quote fragments (`'`, `"`) introduced during prior data serialization are targeted and stripped via sequential character loops.
3. **Basket Deduplication:** To prevent erroneous itemset self-coupling, actors repeated within a single transaction record are flattened using internal set casting before candidate generation blocks execute.

4 Algorithmic Implementation Details

The system targets raw RDD interfaces rather than high-level Spark SQL engines to avoid the memory planning overhead of relational query optimizers:

```
def local_apriori(iterator):
    # Sequential data stream processing within separate cluster worker spaces
    baskets = list(iterator)
    local_counts = {}
    # Internal Apriori combination loops evaluate the partition arrays...
    for candidate in local_combinations:
        if local_counts[candidate] >= local_support_threshold:
            yield candidate
```

Listing 1: Custom Partition-Isolated Candidate Generator

- **Pass 1 Optimization:** Run via `.mapPartitions(local_apriori).distinct().collect()`, forcing worker nodes to isolate intensive memory footprints within local threads.

- **Pass 2 Execution Framework:** The consolidated array is written into a static network broadcast mapping: `spark.sparkContext.broadcast(candidate_list)`. Worker partitions then execute quick dictionary lookups through `.flatMap(count_global_occurrences).reduceByKe a, b: a + b)`, cutting network overhead.

5 Experimental Architecture & Scaling Analysis

5.1 Execution Environment

Testing was deployed on an Apache Spark runtime allocated with 4GB of driver memory allocation. For exact experimental replicability, statistical tracking samples are bound directly to a fixed random seed framework (`seed=42`).

5.2 Performance Metrics

Table **??** outlines execution trends as the system scales from a small, local footprint up to the large, 98k real-world dataset.

5.3 Scalability Evaluation

The infrastructure displays highly resilient scalability. Transitioning to the real-world dataset presents a substantial scaling challenge: the transaction total jumps roughly 10x, and the candidate space grows exponentially to 567,037 itemsets.

Despite this surge, total runtime only moves from 5.68 seconds to 9.29 seconds. This sublinear execution trend underscores the strength of the SON architecture. Because heavy subset processing occurs independently inside isolated partition workers during Pass 1, the cluster avoids expensive global shuffle bottlenecks and balances memory workloads efficiently.

6 Analytical Analysis & Cinematic Results

The final output tables reveal stark, structurally distinct casting distribution patterns between regional film industries. While testing on the sandbox data naturally highlighted common Hollywood pairs (such as *Daniel Radcliffe & Rupert Grint* holding a support of 6), scaling the model to the broader Kaggle footprint shifted the highest support scores entirely toward international film markets.

The top-ranking association belongs to the voice-acting pair **Minami Takayama & Wakana Yamazaki**, marking a true global support score of 19 movies. Similarly, Hong Kong action duos like *Ching Wan Lau & Louis Koo* emerge prominently with an absolute support count of 14.

From a data science standpoint, this dynamic reflects real-world market differences. Hollywood production systems routinely avoid casting identical pairs across large numbers of films due to high talent costs, scheduling constraints, and actor contracts. In contrast, international industries—such as long-running voice-acting anime franchises (*Detective Conan*) or prolific Hong Kong action casting models—rely heavily on long-tail recurring collaborations. The algorithm successfully uncovered these structural dataset characteristics without requiring manual categorization flags.

7 Conclusion

The custom PySpark SON implementation proves highly effective at mining large-scale, messy transaction sets. By decoupling candidate space discovery into distributed partition loops and utilizing driver broadcast variables for counting, the pipeline ensures rapid convergence in under

10 seconds. The system delivers complete algorithmic accuracy, zero false negatives, and robust horizontal scalability.